

面试问答整理

电商 Agent + Vue 前端 + FastAPI + DeepSeek + Java 后端

使用说明：答案按口述风格整理，面试时结合追问自然展开，不需要逐字背诵。

一、AI Agent 项目概述

1、简单介绍你的 AI Agent 电商助手项目整体流程

这是一个 Vue3 + FastAPI Agent + DeepSeek + Spring Boot + MySQL 的智能电商助手。用户在 Vue 页面输入自然语言，前端通过 SSE 请求 FastAPI；Agent 读取会话上下文，让 DeepSeek 判断是直接回答还是调用工具。规则问题走 RAG，商品、购物车和订单走工具调用，再由 Agent 通过 HTTP 请求 Java 后端，Java 完成 JWT、权限、库存、事务校验后访问 MySQL，结果沿原链路返回并渲染成商品卡片或订单卡片。工程上做了写操作二次确认、后端硬校验、有界重试、熔断降级和全链路 Trace ID，本地可以一键启动完整演示。

2、这个电商 Agent 项目是不是对接真实业务系统？

它不是接线上生产系统，而是对接我自己搭建的完整电商业务后端。当前主流程是 API 模式：Agent 通过 HTTP 调用 Java Spring Boot 接口，真实读写 MySQL 里的演示业务数据；另外保留 database/demo 降级能力，方便断网和自动化测试。面试时我会明确说数据是本地 Demo 数据，但前端、鉴权、工具调用、事务和数据库链路都是真实跑通的。

3、项目技术栈都是 Python，Java 是自学的吗？

项目不是纯 Python。Python FastAPI 负责 Agent 编排、模型调用、RAG 和工具调度；Java Spring Boot 负责用户、商品、购物车、订单和支付等确定性业务。我对 Java 是自学加项目实践，能够讲清 Controller、Service、Repository 分层、JWT、事务和悲观锁。开发中有使用 AI 辅助，但接口契约、安全边界和联调问题是我自己理解、验证和修复的，不会把模型生成的代码当黑盒。

4、项目用到 FastAPI 了吗？是不是分移动端和后台管理端？

FastAPI 是中间的 Agent 服务层，不是业务后端。现在有独立 Vue3 用户端，负责登录、聊天、SSE 流式状态和结构化卡片展示；FastAPI 暴露 /chat、/chat/stream、/auth、/confirm 等接口；Java 提供普通用户和管理员 API。当前没有独立移动端，也没有完整的 Admin 管理前端，不能把已有管理接口说成已经做完管理后台。FastAPI 自带的旧网页只保留为兼容和排障入口。

5、项目后端是用什么语言写的？

项目有两层后端。业务后端使用 Java 17 + Spring Boot 3，搭配 JPA、Spring Security、JWT、Flyway 和 MySQL，负责确定性电商业务；AI 后端使用 Python FastAPI，负责 DeepSeek、RAG、工具调用、会话状态和 SSE。Vue 不直接访问模型或数据库，所有聊天先进入 Agent，业务事实再由 Java 返回。

6、你搭的电商底层是什么？

SpringBoot 3 + MySQL 的单体电商后端，三层架构：Controller 收请求、Service 写业务逻辑加事务、Repository 用 JPA 操作数据库。8 张表覆盖用户、商品分类、商品、地址、购物车、订单、订单明细、评价。核心模块是商品、订单、支付、购物车。安全用 JWT 无状态鉴权，登录注册加了 IP 限流防刷。订单是状态机驱动的，0 待支付到 4 已取消。并发靠数据库行锁保证，下单用悲观锁防超卖，取消订单用乐观更新防重复回库存。定时任务自动关超时订单释放库存。

7、讲下你项目的工程化改造，分别解决什么问题？

我主要做了六类改造。第一，写操作二次确认，用一次性 token 绑定 session 和登录身份，防止模型误操作。第二，权限和资源归属下沉 Java Service 硬校验，防止绕过 Agent 越权。第三，工具调用和 HTTP 请求有界重试，只重试超时和 5xx。第四，DeepSeek 异常时自动降级本地规则 Agent，并用熔断器避免持续打故障模型。第五，Vue 使用 SSE 展示执行状态，客户视图和调试视图分开。第六，加入 Trace ID、健康检查和一键启动脚本，方便跨 Vue、Agent、Java 排查问题。

8、项目开发遇到最大的 Bug 是什么，怎么排查解决？

最近最典型的问题是新增 Vue 后看起来完全连不上 Agent。我先从浏览器请求、Vite 代理、Agent 日志、Java 端口逐层排查，发现不是一个点：Java 缺少 flyway-mysql 导致 MySQL 8 启动失败；自定义 TraceldFilter 顺序配置错误；前端用 deep health 把可降级状态误显示成离线；另外 Python httpx 会读取系统代理，把 localhost 的 Java 请求送到代理后返回 502。我分别补依赖、修过滤器顺序、把页面状态改为轻量健康检查，并让本机后端连接绕过环境代理。最后在浏览器里验证 Vue 调用真实 deepseek-v4-pro，再通过工具拿到 Java/MySQL 的商品数据。这个问题让我形成了按链路分层排查，而不是只盯前端报错的习惯。

9、很多代码是 AI 生成的，是不是底层技术自己了解不透彻？

AI 确实辅助过代码生成和重构，但我不是直接复制。我的做法是先定义架构边界和验收标准，再让 AI 处理局部实现，最后通过日志、接口测试、单元测试和浏览器端到端验证。比如 Vue 连不上后端时，我能自己定位 Flyway、过滤器顺序、健康检查和 localhost 代理四个不同层面的问题。面试时我不强调代码是不是手敲，而是证明我能解释设计、发现问题、验证结果并承担最终质量。

10、开发时你使用什么 AI 编码工具？怎么使用？

主要使用过 Codex、Claude 和 Trae 三款 AI 编码工具。Codex 适合快速产出业务代码片段；Claude 上下文窗口大，用来阅读长代码、做代码评审；Trae 作为 AI 原生 IDE，可以读取项目完整代码上下文，辅助做需求拆解、调试排错、生成单元测试。

二、工具调用与安全

11、项目提供了哪些工具函数？

当前一共 14 个工具。知识和商品类 3 个：知识库检索、商品搜索、商品详情；购物车类 5 个：查看、加购、修改数量、删除、清空；订单类 6 个：订单列表、订单详情、取消、退款、创建订单、支付。查询工具可以直接执行；修改购物车、下单、支付、取消和退款等高风险写操作需要登录并进入二次确认。

12、工具是怎么提供给 Agent 使用的？工具列表包含哪些内容？

使用 OpenAI-compatible Function Calling，把工具名称、描述、strict 标记和 JSON Schema 发给 DeepSeek。模型返回工具名和 arguments 后，服务端 ToolRegistry 会再次检查工具白名单、参数类型、登录状态、操作风险和公网只读开关，再决定查询、执行或生成确认令牌。部分明确指令还有确定性规则补强，用来做参数提取和漏调工具纠正，但最终业务修改仍然必须经过服务端校验。

13、工具调用死循环具体是什么情况？

大模型不停调用工具，得不到最终回复。分三类：一是不带新参数重复调用同一个工具；二是多个工具来回互相调用；三是缺少入参，没有补充参数就直接重试请求。

14、工具调用设置最大调用次数上限的作用是什么？是循环跑 5 次减少误差吗？

不是固定循环 5 次降低误差，而是最多允许 5

轮模型与工具交互。模型可能先查商品、再查详情，或者根据工具结果继续决策；如果一直重复调用就会消耗 token 并拖垮请求。达到上限后停止继续调用，使用已有结果生成回答或返回明确错误。这个上限通过 MAX_TOOL_ROUNDS 配置。

15、工具调用达到上限 / 持续失败怎么处理？

设置可配置的最大工具调用轮次,到达上限直接终止执行,返回提示告知用户请求步骤过多,同时保留已经执行完成的工具结果。工具执行失败时,仅对超时、5xx 这类可恢复异常最多重试 3 次(指数退避),业务参数错误、鉴权失败直接返回不重试。

16、为什么权限校验不能交给大模型,必须后端硬校验？

大模型存在幻觉问题,有可能绕过提示词约束做出错误权限判断。权限属于业务安全逻辑,需要确定性代码保证结果,放到后端 Service 层硬校验,无论大模型输出什么,都必须经过后端校验,从根源避免越权风险。

17、如何结合规则、RAG 和工具调用减少大模型幻觉？

我把数据分三类处理：普通闲聊允许模型直接回答；售后规则和 FAQ 必须先检索

RAG；价格、库存、购物车、订单必须调用 Java

工具拿实时数据。模型只负责理解、选择和组织语言，不能作为业务数据源。服务端还会对工具参数做 Schema 校验、意图守卫和权限校验，Java 再做最终业务校验。即使模型说库存足够，Java 数据库校验不通过也不会执行。

18、举例说明你定义了哪些业务规则？

多层业务约束:第一层下单加购校验库存;第二层单个商品最多购买 99 件;第三层 5 分钟内防止重复下单;第四层只有待支付订单允许取消;第五层写操作强制二次确认;第六层工具守卫,禁止大模型无理由调用工具。

19、LangGraph 是什么？你在项目里怎么使用？

LangGraph 是带状态的工作流编排框架，适合循环、分支、持久化和

Human-in-the-Loop。当前项目没有把所有模型工具循环都包装成 LangGraph 图，核心模型工具循环由

AgentService 控制；LangGraph 主要用于高风险操作审批。工具先生成待确认状态，图在 interrupt()

暂停，用户通过 /confirm 明确同意后再用 Command(resume=approved)

恢复执行。这样说和当前代码一致，也避免把普通 while 工具循环夸成完整 LangGraph Agent。

20、LangGraph 在这个 AI 应用里主要解决什么问题？

主要解决高风险写操作的可暂停、可恢复和状态持久化问题。下单、支付、取消、退款等操作不能让模型直接执行，所以流程先暂停等待用户确认。配置 Redis 时使用 Redis Checkpointer，服务多实例或重启后更容易恢复；没有 Redis 时回退 InMemorySaver，功能还能用，但进程重启会丢失待确认状态。

21、LangGraph 如何联动完成高危订单二次确认？

第一步，模型提出取消或支付等工具调用；ToolRegistry 校验登录身份、订单归属和参数，只生成一次性确认

token，不修改数据库。第二步，Vue 展示确认卡，用户点击后调用 /confirm，携带 session_id、JWT 和

token。第三步，Agent 原子消费 token，检查 token 绑定的 session 和 JWT 指纹，再用 LangGraph resume

恢复。第四步，真正调用 Java 接口，Java 再做 JWT、归属和订单状态校验，并在事务中修改

MySQL。拒绝或过期都不会执行。

22、如果用户超时没有确认，系统怎么处理？

当前确认 token 默认有效期 60 秒。待确认数据保存在内存或 Redis，过期后 /confirm 无法消费，用户需要重新发起。token 只能原子消费一次，并绑定 session_id 和 JWT 指纹，所以复制 token、换账号确认或重复提交都不能执行。不是靠前端倒计时保证安全，真正的过期和防重放判断在 Agent 服务端。

23、高危订单确认流程还能怎么优化？

现有流程已经做了身份绑定、一次性 token、60 秒 TTL、服务端原子消费和 Java 二次校验。后续优先优化审计与可观测性：把发起人、操作类型、业务对象、确认结果和 Trace ID 统一记录；多实例部署强制使用 Redis Checkpointer 和 Redis 确认存储；对高金额或退款操作再增加风险等级和管理员审批。

三、RAG 知识库

24、大模型幻觉底层为什么会发生？项目做了哪些手段抑制？

底层原因：大模型本质是基于统计概率逐 token 预测输出，优先保证语句通顺，不知道答案也会生成看起来合理的内容；训练数据集本身存在错误老旧信息；RAG 检索到错误文档片段也会诱导幻觉。项目中通过 RAG 真实知识库、工具调用读取真实业务数据、后端校验、二次确认减轻幻觉。提示词约束、RAG、工具调用只能减轻幻觉，没有办法彻底根除。

25、项目里的知识库 RAG 是做什么的？

知识库分 rule 和 product 两个隔离库。rule 存退款、物流、质保等静态规则；product 存品牌、型号、规格和卖点等非实时属性。价格、库存、订单状态绝不从知识库回答，必须调用 Java 业务接口。知识优先从 MySQL 加载，没有数据时回退本地 JSON，因此数据库临时不可用也能保留基础问答能力。

26、一条规则问题怎么走完整 RAG 流程？

例如用户问七天无理由退货。Agent 先根据意图路由到 rule 库；知识库把问题分词并做同义词扩展，用 TF-IDF 计算关键词相关度。如果配置了独立 Embedding 服务，再计算向量相似度，与 TF-IDF 做 Hybrid Search；配置 Rerank 服务时可继续重排，最后取 Top-K 片段。检索结果会过滤提示注入标记，再连同用户问题交给 DeepSeek 组织回答。Embedding 或重排失败会自动降级纯 TF-IDF，不影响主流程。

27、做 RAG 为什么需要向量化？项目一定依赖向量库吗？

向量能解决同义词和改写后的语义召回，但也可能召回语义接近却业务无关的内容。当前项目支持可选 Embedding，与 TF-IDF 以 0.5 比 0.5 融合，但不强依赖独立向量数据库；文档规模较小时可以在内存中计算并缓存向量。没有 Embedding Key 就降级纯 TF-IDF，所以本地演示和自动测试仍然稳定。

28、如何通过知识库限制 Agent 调用工具？

实现工具守卫函数，根据用户关键词限制工具调用时机；同时系统提示词明确区分职责，静态售后规则走知识库，价格库存这类实时数据调用商品工具，两层约束大模型行为。

29、商品业务数据如何更新，避免返回过时数据？

商品知识库只保存品牌、规格、卖点等相对稳定信息；价格、库存、上下架状态全部走 search_products 或 get_product_detail 实时请求 Java/MySQL。知识库支持管理接口重新加载和增量重建，商品表更新不会靠模型记忆。这样既能用 RAG 做语义说明，又不会拿旧知识片段回答实时库存。

30、简历上关于向量库的，实际项目是怎么接入的？

当前项目实现的是可降级 Hybrid RAG：有 Embedding 配置时生成向量并缓存，和 TF-IDF 融合；没有配置时使用 TF-IDF 加同义词扩展。项目没有把 FAISS、Milvus 或 Pgvector 作为强依赖，所以我不会说已经做过大规模向量数据库调优。这个选择符合当前百条级知识规模，优点是部署简单、可解释、故障可降级。

四、会话状态与并发

31、项目怎么存储会话状态？为什么聊天记录和业务上下文分开？

会话键由 session_id 加 JWT 的 SHA-256 指纹组成，避免不同账号共用一个浏览器会话时串数据，也不明文存 token。聊天记忆保存最近消息并在达到阈值后压缩旧内容；结构化状态单独保存最近商品、订单和购物车引用，用来解析“刚才那个”。配置 Redis 时支持 TTL 和多实例共享；未配置时回退进程内存，默认 30 分钟过期。

32、项目是双进程处理 Agent 任务吗？

当前不是固定双进程架构。FastAPI 使用 async/await 处理模型和 HTTP 这类 IO 密集任务，SSE 请求内部用异步任务执行 Agent，同时定期向前端推送进度。生产环境可以用多个 worker 或多实例扩容，会话和确认状态需要放 Redis；本地开发一般一个 Agent 进程即可。面试时不会把异步协程错误描述成双进程。

33、用户说“帮我取消刚刚那个订单”这种模糊指令，怎么处理？

Agent 从独立的结构化会话状态读取最近展示或操作过的订单 ID，并返回 reference 信息让 Vue 和调试面板看到解析依据。如果只能找到一个明确对象就继续；如果存在多个候选或没有上下文，就追问用户，不能猜订单号。取消前还要查询 Java 确认订单属于当前用户且状态允许取消，再生成确认 token。

34、如果 DeepSeek 服务直接挂掉，系统还能不能工作？

AGENT_MODE=auto 时每次优先调用真实 DeepSeek；出现连接失败、超时或熔断后，本次请求可以降级到本地规则 Agent，完成商品查询、购物车和订单等基础能力。健康接口会把 serving_mode 标成 demo_fallback，Vue 显示“降级可用”，而不是伪装成真实模型在线。模型恢复后成功请求会重新回到 live。降级保证核心可用，但复杂自然语言理解能力会下降。

35、查询等功能是不是并行执行的？采用什么方式？

Vue 到 Agent 使用 SSE 长连接；FastAPI 用异步 IO，在等待 DeepSeek 或 Java HTTP 时不会阻塞整个进程；Spring Boot 由 Tomcat 线程池并发处理请求。业务并发安全不能靠线程模型，下单仍然要靠数据库事务和悲观锁防超卖。RAG 单次检索主要是内存计算，不会为了形式上并行而增加复杂度。多实例时会话和确认状态放 Redis，超时关单使用数据库锁避免重复处理。

36、悲观锁如何保证并发数据稳定？

悲观锁默认认为一定会发生多人抢改同一条数据。比如多人同时扣同一个商品库存，只要有一个人开始操作这条数据，直接上锁，其他人必须等操作完成解锁才可以修改，避免同时修改出现库存算错、订单错乱。MySQL 写法：SELECT ... FOR UPDATE。

五、电商后端与数据库

37、项目数据库核心表怎么设计?

8 张业务表: users (用户, role 区分普通用户 / 管理员)、 categories (商品分类)、 products (商品, 含 stock 库存、 soldCount 销量)、 addresses (收货地址)、 shopping_carts (购物车)、 orders (订单, status 状态机 0 待支付 → 4 已取消, paymentStatus 支付状态)、 order_items (订单明细, 冗余商品名称价格快照)、 reviews (商品评价)。

38、订单表, 写 SQL 按天统计每一天订单数量

```
SELECT DATE(create_time), COUNT(*) FROM `order` GROUP BY DATE(create_time);
```

DATE(create_time) 只拿出日期丢掉时分秒; GROUP BY 按日期分组; COUNT(*) 统计每组订单行数。

39、Docker 容器部署常用命令

把项目打包成独立 " 软件包 ", 在别的服务器直接运行。

docker ps: 查看正在运行容器

docker run: 启动容器

docker logs: 查看容器运行日志

六、Java 基础

40、谈谈你对 SpringBoot 的理解

SpringBoot 是基于 Spring 的快速开发框架, 实现自动配置, 省去大量 XML 配置, 内置 web 容器, 可以快速开发 REST 接口。项目使用 MVC 分层架构, Controller 接收请求, Service 封装业务逻辑, Mapper 操作数据库; 可以便捷整合 MySQL、Redis 等各类组件。

41、MVC 分层架构, Controller、Service、Mapper 职责? 业务权限校验放哪一层? 为什么?

Controller 负责接收 HTTP 请求, 解析请求参数; Service 封装核心业务逻辑, 权限、越权校验都放在 Service 层; Mapper 负责和数据库交互; Model 实体映射数据库表。

权限校验放在 Service 层, 无论调用方是前端还是 AI Agent, 所有请求都会经过业务层, 保证校验逻辑不会被绕过。

42、@Transactional 事务注解是做什么的? 什么场景会失效?

@Transactional 是 Spring 声明式事务, 用来管理数据库事务, 保证一组数据库操作原子性, 异常自动回滚。

失效场景: 方法不是 public; 业务代码内部 catch 捕获吃掉异常, 框架感知不到异常, 不会触发回滚。

43、ArrayList 和 LinkedList 区别, 平时优先用哪个?

ArrayList 底层动态数组, 支持随机访问, 查询效率高; 中间插入删除需要移动大量元素, 性能差。LinkedList

底层双向链表, 随机访问需要遍历, 查询效率低; 头尾节点增删效率高。绝大多数业务开发优先选择

ArrayList。

44、HashMap 底层原理? 为什么线程不安全? 并发场景用什么?

HashMap 底层是数组 + 链表结构, 哈希冲突时同一个下标形成链表; 链表超过阈值转为红黑树提升查询性能。

HashMap 没有锁机制, 多线程并发修改会出现数据丢失, 线程不安全。并发场景推荐 ConcurrentHashMap。

45、Java 多线程有哪几种实现方式?进程和线程区别?

三种实现:1继承 Thread 类重写 run() ;2实现 Runnable 接口交给 Thread 执行;3实现 Callable 接口,可以拿到线程执行返回结果。

进程是操作系统资源分配最小单位;线程属于进程,同一个进程内多个线程共享堆内存,线程切换开销比进程小。

46、什么是线程不安全? synchronized 作用?

多个线程并发修改同一个共享变量,出现数据错乱,就是线程不安全。synchronized 提供同步锁,保证同一时刻只有一个线程执行被锁住的代码,解决并发线程安全问题。

47、try-catch-finally 分别作用? finally 一定执行吗?

try 包裹可能抛出异常的业务代码; catch 捕获对应异常做业务容错处理; finally 无论是否抛出异常都会执行,一般用来关闭流、释放连接资源。

48、什么是 RESTful API 接口规范?

RESTful 是一套接口设计风格。使用 HTTP 动词代表操作行为: GET 查询资源, POST 新增, PUT 更新, DELETE 删除; URL 使用名词;使用 HTTP 状态码标识请求结果,返回统一响应格式。

49、GET、POST、PUT、DELETE 的区别

GET :获取查询数据,参数放 URL ,不修改服务端数据; POST :新增创建资源,请求体携带参数; PUT :完整更新已有资源; DELETE :删除对应资源。

50、简单讲讲 SpringCloud ,微服务会带来哪些问题?

SpringCloud 是一套微服务组件集合,基于 SpringBoot ,提供服务注册发现、网关、配置中心等组件,用来管理拆分后的微服务。微服务会遇到远程调用超时失败、分布式事务、服务治理等问题。我主要熟练 SpringBoot ,了解微服务概念,工作中可以快速学习上手。

51、MCP 是什么?

MCP 是一套工具调用的标准化协议,统一工具描述、入参输出格式。好处是 Agent 可以快速对接各类工具,不用每个工具单独写适配代码;但 MCP 只解决通信标准,权限、业务校验依旧要后端自己处理,协议本身不会保障业务安全。

七、数据库 & Redis

52、索引是什么?优缺点?

索引就像数据表目录,用来加速查询。优点:大幅提升 select 查询速度;缺点:占用磁盘存储空间;新增修改删除数据时索引需要同步更新,降低写操作性能。

53、where 和 having 的区别?

where 是 group by 分组之前过滤原始数据,不能使用聚合函数; having 是分组完成之后对分组结果过滤,可以使用 count、sum 这类聚合函数。

54、内连接、左外连接区别？

内连接 INNER JOIN :只返回两张表关联条件匹配成功的数据,两边都匹配到才出结果。

左外连接 LEFT JOIN :以左表为主,左表全部数据保留,右表能匹配就带出,匹配不到右表字段为 null 。

右外连接 RIGHT JOIN :以右表为主,右表全部数据保留。

全外连接 FULL JOIN :左右两张表全部数据都保留。 MySQL 不支持全外连接语法,需要用 union 合并左右连接结果模拟。

内连接没有左右之分;带左 / 右的都是外连接。

55、MySQL 和 PostgreSQL 简单区别？

都是关系型数据库,基础增删改查语法大体一致。 PostgreSQL 对 JSON 数据、复杂统计查询支持能力更强;日常主要使用 MySQL ,可以快速适配 PostgreSQL 。

56、数据库连接池是什么?为什么需要?

数据库连接是程序和数据库通信通道。频繁创建销毁连接开销巨大;连接池提前初始化一批连接,业务需要时直接复用,减少资源开销,提升数据库访问效率。

57、Redis 为什么速度快? 有哪些基础数据类型? 项目怎么使用?

Redis 主要在内存中读写, 数据结构操作复杂度低, 并通过事件循环高效处理网络请求。基础类型有 String、List、Hash、Set、ZSet。项目中 Redis 是可选基础设施: 保存聊天记忆、结构化会话状态、一次性确认 token 和 LangGraph Checkpoint, 并设置 TTL。Redis 不可用时回退内存实现, 功能还能跑, 但不能跨进程共享, 重启后状态会丢失。

八、中间件

58、Kafka 是什么,作用?

Kafka 是消息中间件消息队列。用于异步解耦、流量削峰,服务之间异步传递消息。了解概念,缺少深度实战,工作需要可以快速学习。

59、Elasticsearch (ES)是干什么的?

Elasticsearch 是全文检索组件,擅长海量文本数据检索,实现搜索能力。了解基础概念,项目没有深度使用,工作中可以查阅文档快速上手。

60、Nginx 的作用? 项目里怎么使用?

Nginx 是生产环境前端静态资源服务器和反向代理。它把 Vue 页面发给浏览器, 再把 /chat、/auth、/confirm 等请求转发到 FastAPI Agent; 对 SSE 关闭代理缓冲, 保证进度事件及时返回; 还可以做 IP 限流、TLS、访问日志和隐藏 /admin、/metrics。开发环境没有 Nginx 时由 Vite proxy 完成同样的同源转发。

九、Linux / HTTP / 通用

61、Linux 常用命令

ls 查看目录文件; cd 切换目录; pwd 查看当前路径; mkdir 创建文件夹; rm 删除文件; cat 查看文件内容; ps 查看进程; kill 关闭进程; chmod 修改文件权限。

62、请求头、请求体在这个项目里分别放什么？

请求头放协议和身份元信息，比如 Authorization: Bearer JWT、Accept: text/event-stream、Content-Type、X-Trace-Id；请求体放业务参数，比如 message、session_id、confirmation_token。JWT 正常放请求头，不用 IP 或 session_id 代替身份。Agent 调 Java 时会继续转发 JWT 和 Trace ID，便于鉴权和全链路排查。

63、常见 HTTP 状态码

200 请求成功; 404 资源找不到; 401 未授权, token 失效; 403 权限不足; 500 服务端内部异常。

64、什么是跨域？这个项目如何解决？

协议、域名或端口不同就属于不同源，浏览器会按同源策略限制请求。开发环境中 Vue 在 5173、Agent 在 8000，但前端只请求相对路径，由 Vite proxy 转发到 Agent，所以浏览器看到的是同源请求；生产环境由 Nginx 反代。FastAPI 仍配置明确的 CORS 白名单作为补充，生产环境不能直接使用星号放开所有来源。

65、ORM 是什么？有什么好处？

ORM 对象关系映射,把实体类映射数据库表,通过面向对象的方式操作数据库,减少手写原生 SQL,提升开发效率,降低 SQL 拼写错误。

66、Cookie 和 Session 区别

Cookie 存储在浏览器客户端; Session 存储服务端。Cookie 有大小限制, Session 相对更安全。

67、Git 分支 workflow 怎么使用？

开发新建独立开发分支;本地编码自测完成提交推送远程;提交 PR 进行检查,验证通过合并 main 主分支;使用 .gitignore 忽略密钥、配置文件,避免敏感信息提交仓库。

十、HR 问答

68、做个自我介绍

面试官您好，我叫郭锦鸿，是 27 届数据科学与大数据技术本科。我的方向是 AI 应用和全栈工程，熟悉 Python FastAPI、Vue3、Java Spring Boot、MySQL 和 Redis。我独立完成了智能电商 Agent：Vue 负责用户交互和 SSE，FastAPI 接入 DeepSeek 做 RAG、工具调用和会话编排，Java 负责 JWT、订单、库存和事务，MySQL 保存真实业务数据。我针对幻觉、越权、模型故障、工具超时和高风险写操作做了确定性防护，并把项目做成可一键启动、可端到端演示的完整链路。虽然企业实习经历还不多，但我有比较完整的排障和工程落地经验，希望应聘 AI 应用开发或 AI 全栈岗位。

69、为什么投递 AI 应用开发或 Java AI 应用开发岗位？

这个岗位既要求把模型能力接进业务，也要求后端工程可靠性，和我的项目经历匹配。我不仅调用过模型 API，还做过 Vue 交互、SSE、Function Calling、RAG、JWT、Java 业务接口、MySQL 事务、重试降级和监控排障。我希望把个人项目里形成的安全边界和全链路思维用到真实团队，同时继续学习工业级微服务、评测和向量检索。

70、没有企业正式实习经历，怎么保证胜任？

我会用可验证的项目结果证明能力：项目不是一段 Prompt，而是 Vue、Agent、Java、MySQL 的完整系统；我能从浏览器请求一路排查到代理、Python 客户端、Java 启动和数据库；每次修改会补测试并做真实浏览器验证。我知道个人项目和生产系统仍有差距，所以不会夸大经验，但需求明确后我能快速拆分、查官方文档、完成实现并对结果负责。

71、怎么理解 AI 应用开发？

AI 应用开发不是只写提示词，而是把不确定的大模型安全地接入确定性的业务系统。模型负责理解和生成，工具负责获取事实，后端负责权限、事务和数据一致性；外围还需要评测、日志、追踪、限流、重试、熔断、降级和人工确认。目标不是让模型看起来聪明，而是在错误和故障发生时系统仍然可控。

72、在 AI 应用开发中，你觉得哪块模块你能快速上手？

我能较快上手 Agent 工具调用、RAG、FastAPI 接口、Vue 对话交互以及和 Java 业务系统联调，因为这些我做过完整链路。对于大规模向量数据库调优、模型微调和复杂 Kubernetes 运维，我目前实践较少，会如实说明并通过官方文档和小规模验证快速补齐。

兜底万能句

项目次要细节

当时主要侧重把功能落地实现，这部分细节我现在记不太清楚了。

陌生知识点

不好意思，这块知识点我目前还没有深入学习。如果工作需要用到，我会查阅官方文档，先做小规模验证，再快速补齐上手。

十一、Vue 前端与全链路

73、为什么后来新增 Vue 前端？

旧页面由 FastAPI 静态托管，能演示但组件化、状态管理和扩展性有限。新增 Vue3 后把聊天、登录、商品卡片、订单卡片、确认卡片和调试面板拆成组件，使用 Pinia 管理会话和登录状态，TypeScript 约束接口数据。这样用户体验更完整，也方便后续独立部署和团队协作。

74、Vue 如何把一条消息发给 Agent？

Vue 先生成 session_id，并从 Pinia 读取可选 JWT，然后 POST /chat/stream。开发环境请求先到 Vite proxy，再转发 FastAPI 8000 端口；生产环境由 Nginx 反代。请求体包含 message 和 session_id，JWT 放 Authorization 请求头。前端持续解析 SSE 的 started、progress、tool、confirmation_required、result 和 error 事件，更新同一条助手消息。

75、为什么聊天流式返回使用 SSE，不用 WebSocket？

当前场景主要是服务器向浏览器单向推送 Agent 进度和最终结果，客户端发消息仍然用普通 HTTP，所以 SSE 更轻量，天然兼容 HTTP、鉴权和反向代理。WebSocket 适合高频双向通信，但需要处理连接升级、心跳和重连。项目使用 SSE 已能满足“正在理解、调用工具、返回结果”的实时反馈。

76、IP、JWT、session_id、Trace ID 分别有什么作用？

IP 用于限流和审计，不能代表用户；JWT 代表登录身份和权限；session_id 区分一段对话，但不能代替鉴权；Trace ID 用于串联 Vue、Agent 和 Java 日志。查看订单既要 JWT，还要 Java 校验订单归属；同一个 IP 下可能有多个用户，所以不能用 IP 判断资源属于谁。

77、前端显示离线就一定是 Vue 请求没发出去吗？

不一定。要分层排查：浏览器 Network 看请求是否发出；Vite/Nginx 看代理；Agent /health 和日志看服务状态；再检查 Java 和 MySQL。项目曾经是 Java 启动失败、系统代理劫持 localhost、深度健康检查误判同时存在。最终要用真实问题验证：页面显示模型在线，DeepSeek 产生工具调用，Java 日志收到请求，Vue 渲染数据库商品，不能只看一个绿色状态。

78、举例说明推荐 3000 元以内手机的完整链路

用户在 Vue 输入问题，前端通过 SSE 发给 Agent；DeepSeek 提取 keyword=手机、max_price=3000、in_stock=true 并调用 search_products；ToolRegistry 校验参数；EcommerceClient 请求 Java /api/products/search；Java 查询 MySQL 并返回结构化商品；DeepSeek 只能根据工具结果组织推荐；Vue 把文本和商品列表渲染为卡片。这个查询是公开只读操作，不需要登录。

79、登录和受保护操作怎么联动？

Vue 调 /auth/login，Agent 把账号密码转发 Java /users/login，Java 校验后签发 JWT，Agent 原样返回。之后 Vue 把 JWT 放 Authorization 请求头。Agent 会在 SSE 开始前验证登录，工具注册器检查受保护工具，Java 再做最终 JWT 和资源归属校验。JWT 过期时 Vue 清理本地状态并提示重新登录。

80、用户端为什么不应该显示所有 Agent 技术细节？

普通用户只关心是否在处理、结果是什么、是否需要确认，不需要看到函数名、JSON 参数、模型供应商和内部错误堆栈。当前 Vue 已区分对话视图和调试视图，后续会进一步默认隐藏开发信息，只保留友好状态；调试面板用于开发和面试演示，并对 token、密码等敏感内容脱敏。